

Analisis de Estructuras de Datos

Supermarket Catalog Management - Fase 2

EDD 2026-1S | USAC CUNOC

Lenguaje: Java 21 | Build: Maven 3.8+ | GUI: JavaFX 21

6 sucursales | ~1050 productos | 17 conexiones de red

Indice de Contenidos

1. Diseño de las Estructuras de Datos
 - 1.1 LinkedList - Lista Enlazada Simple
 - 1.2 AVLTree - Arbol Auto-balanceado
 - 1.3 BTree (T=3) - Arbol B por Fecha
 - 1.4 BPlusTree (Orden=4) - Arbol B+ por Categoria
 - 1.5 HashTable (djb2) - Tabla Hash por Barcode
 - 1.6 Stack (LIFO) - Pila para Rollback
 - 1.7 Queue (FIFO) - Cola para Simulacion
 - 1.8 BranchGraph + MinHeap - Grafo + Dijkstra
 - 1.9 Catalog - Coordinador de Estructuras
 - 1.10 Arquitectura General del Sistema
2. Justificacion de Big-O
3. Metodologia de Medicion
4. Resultados (Tablas y Graficas)
5. Conclusiones

Este reporte analiza las 9 estructuras de datos implementadas desde cero en el proyecto de gestion de catalogo de supermercado, incluyendo su diseño, complejidad asintotica, metodologia de medicion de rendimiento y resultados empiricos.

1. Diseno de las Estructuras de Datos

El proyecto implementa un sistema de gestion de catalogo de supermercado con multiples sucursales. Cada sucursal mantiene un Catalog que coordina 5 estructuras de datos principales (LinkedList, AVLTree, HashTable, BTree, BPlusTree) mas un Stack para rollback. A nivel global, un BranchManager coordina todos los catalogos y utiliza un BranchGraph con Dijkstra para calcular rutas optimas de transferencia entre sucursales. Un Dispatcher simula el flujo de productos usando colas Queue.

1.1 LinkedList - Lista Enlazada Simple NO Ordenada

Implementacion de lista enlazada simple con nodo que contiene un Product y referencia al siguiente. La insercion se realiza al frente en $O(1)$. La busqueda (por barcode o nombre) y eliminacion requieren recorrer la lista completa en $O(n)$. Se usa como baseline de comparacion en benchmarks y para iteracion sobre todos los productos del catalogo.

Archivo: structures/LinkedList.java | Nodo interno: Node(Product data, Node next)

LinkedList - Lista Enlazada Simple



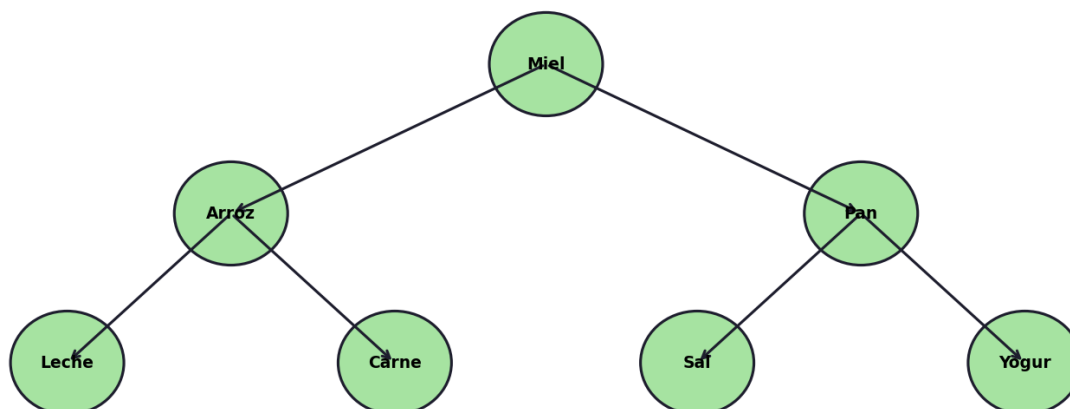
Insert: $O(1)$ al frente | Search: $O(n)$ | Delete: $O(n)$

1.2 AVLTree - Arbol Binario Auto-balanceado por Nombre

Arbol AVL ordenado por nombre de producto (comparacion case-insensitive). Cada nodo mantiene su altura para calcular el factor de balance. Implementa 4 rotaciones: LL (rotacion derecha simple), LR (izquierda-derecha), RR (rotacion izquierda simple), RL (derecha-izquierda). Las operaciones de busqueda, insercion y eliminacion son $O(\log n)$. El recorrido in-order produce los productos en orden alfabetico.

Archivo: structures/AVLTree.java | Nodo: Node(Product, left, right, height)

AVLTree - Arbol Balanceado por Nombre



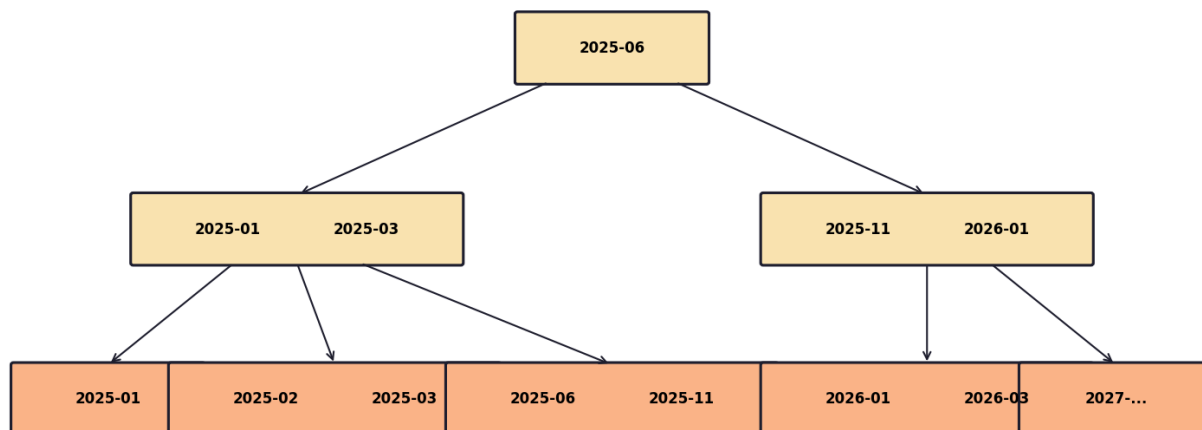
Insert/Search/Delete: $O(\log n)$ | In-order: $O(n)$ | Rotaciones: LL, LR, RR, RL

1.3 BTree (T=3) - Arbol B por Fecha de Caducidad

Arbol B de parametro minimo T=3 (maximo 5 claves por nodo). Indexado por fecha de caducidad en formato ISO 'YYYY-MM-DD' con comparacion lexicografica. Implementa split de hijos durante la insercion y operaciones de borrow/merge durante la eliminacion para mantener las invariantes del arbol B. Soporta busquedas por rango de fechas eficientemente.

Archivo: structures/BTree.java | Nodo: BNode(keys[5], products[5], children[6], n, isLeaf)

BTree (T=3) - Indexado por Fecha de Caducidad



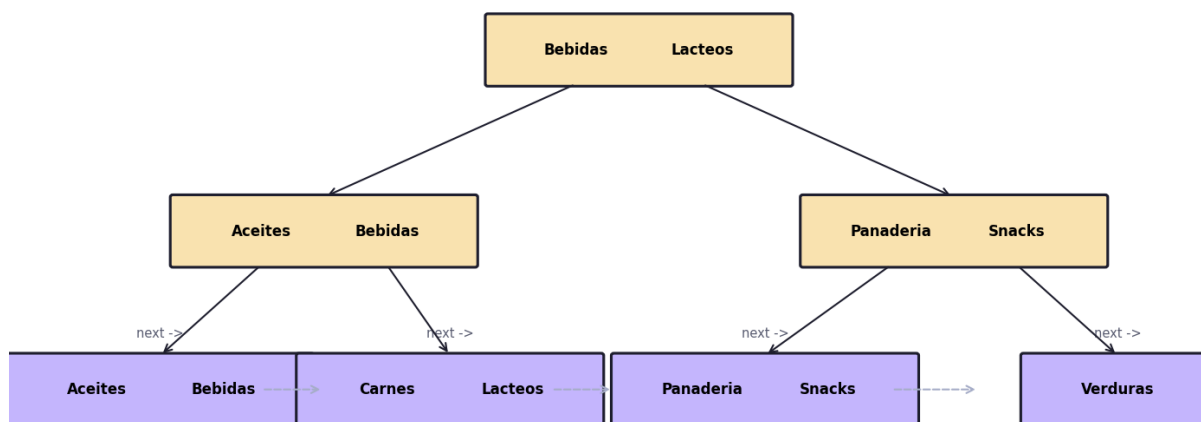
Insert/Search/Delete: $O(T * \log_T n)$ | Range Search: $O(T * \log_T n + k)$ | $T=3$

1.4 BPlusTree (Orden=4) - Arbol B+ por Categoria

Arbol B+ de orden 4 donde solo las hojas almacenan registros de productos. Los nodos internos contienen solo claves de separacion. Las hojas estan enlazadas mediante un puntero 'next', permitiendo recorrido secuencial eficiente por categoria en $O(n)$. Ideal para queries que requieren todos los productos de una categoria.

Archivo: structures/BPlusTree.java | Nodos: LeafNode(keys[4], records[4], next) + InternalNode(keys[4], children[5])

BPlusTree (Orden=4) - Indexado por Categoria



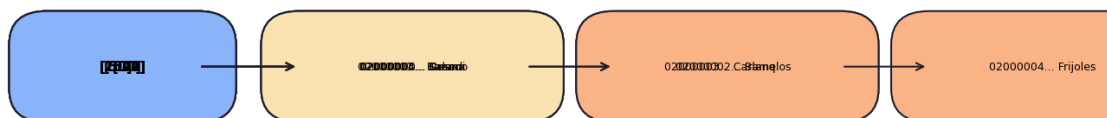
Insert/Search: $O(\log n)$ | Category Search: $O(\log n + k)$ | Leaf chain traversal: $O(n)$

1.5 HashTable (djb2, cap=2048) - Tabla Hash por Barcode

Tabla hash con encadenamiento separado (separate chaining) usando listas enlazadas. Funcion hash djb2: $h = ((h \ll 5) + h) + c$, con seed inicial 5381. Capacidad fija de 2048 buckets. La clave es el barcode del producto, que es unico globalmente. Operaciones de insercion, busqueda y eliminacion en $O(1)$ promedio. El peor caso $O(n)$ ocurre cuando todos los productos colisionan en un mismo bucket.

Archivo: structures/HashTable.java | Entrada: Entry(Product data, Entry next)

HashTable (djb2, cap=2048) - Acceso por Barcode



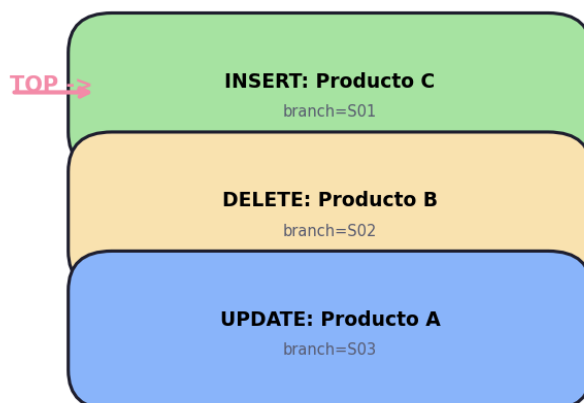
Hash: djb2 ($h = ((h \ll 5) + h) + c$, seed=5381) | Insert/Search/Delete: $O(1)$ avg, $O(n)$ worst

1.6 Stack (LIFO) - Pila para Rollback/Undo

Pila LIFO basada en lista enlazada con tamaño máximo configurable (default 200). Cada entrada registra el tipo de operación (INSERT, DELETE, UPDATE), un snapshot del producto antes del cambio, y el ID de la sucursal. Cuando se alcanza el máximo, se descarta la entrada más antigua con `removeBottom()` en $O(n)$. Se usa para deshacer operaciones del catálogo.

Archivo: structures/Stack.java | Entrada: Entry(Op, Product snapshot, String branchId)

Stack (LIFO) - Rollback/Undo



Push/Pop/Peek: $O(1)$ | removeBottom: $O(n)$ | maxSize=200

1.7 Queue (FIFO) - Cola para Simulacion de Despacho

Cola FIFO con punteros a head (frente) y tail (final) para operaciones $O(1)$. Cada sucursal tiene 3 colas en el Dispatcher: Ingreso (recepcion de productos), Traspaso (preparacion de transferencias), y Salida (despacho). Permite eliminacion por barcode en $O(n)$ para casos especiales.

Archivo: structures/Queue.java | Nodo: Node(Product data, Node next)

Queue (FIFO) - Simulacion de Despacho



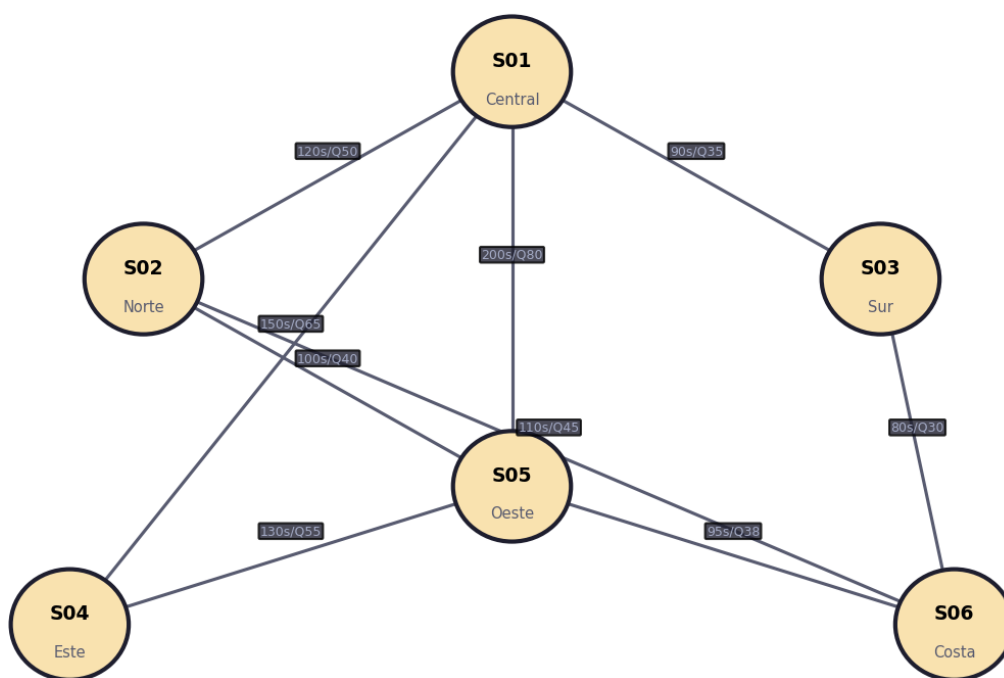
Enqueue/Dequeue/Peek: $O(1)$ | Remove by barcode: $O(n)$

1.8 BranchGraph + MinHeap - Grafo Ponderado + Dijkstra

Grafo ponderado que modela la red de sucursales. Nodos: 6 sucursales. Aristas: 17 conexiones (mayormente bidireccionales) con doble peso: tiempo (segundos) y costo (monetario). Implementado con lista de adyacencia usando nodos enlazados. El algoritmo de Dijkstra utiliza un MinHeap manual (array-based binary heap) sin depender de PriorityQueue de Java. El criterio de optimizacion puede ser tiempo o costo.

Archivo: graph/BranchGraph.java | MinHeap: HeapEntry(nodeld, dist) con siftUp/siftDown

BranchGraph - Red de Sucursales + Dijkstra



Dijkstra con MinHeap: $O((V+E) * \log V)$ | $V=6$ sucursales, $E=17$ conexiones

1.9 Catalog - Coordinador de Estructuras por Sucursal

El Catalog es el coordinador central de cada sucursal. Integra las 5 estructuras de datos y garantiza consistencia mediante insercion atomizada con rollback: si falla la insercion en alguna estructura, se revierten las inserciones

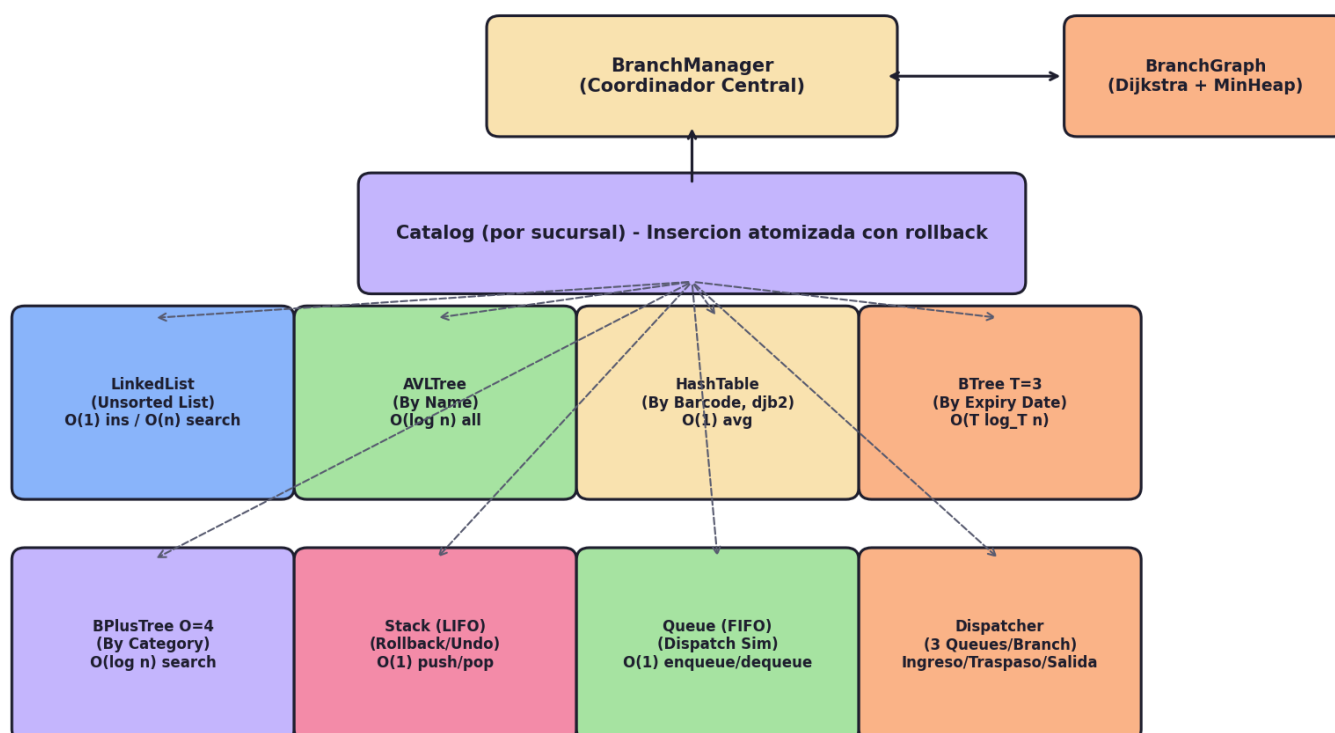
previas. La eliminacion se propaga a todas las estructuras. El Stack de undo permite revertir la ultima operacion (INSERT -> se elimina, DELETE -> se re-inserta).

Archivo: catalog/Catalog.java | Estructuras: LinkedList + AVLTree + HashTable + BTree + BPlusTree + Stack

1.10 Arquitectura General del Sistema

El BranchManager coordina todos los Catalogs (LinkedHashMap por branch ID) y el BranchGraph. El Dispatcher crea 3 Queues por sucursal y ejecuta simulaciones en threads daemon. La GUI JavaFX proporciona vistas para dashboard, CRUD de sucursales/productos, transferencias, visualizacion de grafos/arboles, y benchmarking.

Arquitectura del Sistema - Catalog por Sucursal



2. Justificacion de Big-O

Cada estructura de datos fue seleccionada para optimizar un tipo especifico de operacion. A continuacion se justifica la complejidad asintotica de cada una basandose en su implementacion.

2.1 LinkedList - $O(1)$ insercion, $O(n)$ busqueda/eliminacion

INSERCIÓN $O(1)$: Se inserta siempre al frente de la lista. Solo requiere crear un nuevo nodo y actualizar dos punteros ($n.next = head$; $head = n$), sin importar el tamaño n de la lista.

BUSQUEDA $O(n)$: En el peor caso (elemento al final o no existente) se deben recorrer todos los n nodos, comparando cada barcode o nombre. No hay ordenamiento que permita podar la busqueda.

ELIMINACIÓN $O(n)$: Requiere primero encontrar el nodo ($O(n)$) y luego ajustar punteros. En el peor caso se recorre toda la lista.

2.2 AVLTree - $O(\log n)$ para todas las operaciones

El AVL es un árbol binario de busqueda auto-balanceado donde la diferencia de alturas entre sub-árboles izquierdo y derecho (factor de balance) es como máximo 1 en cada nodo.

Altura garantizada: $h \leq 1.44 * \log_2(n+2) - 1.32$, es decir $O(\log n)$.

INSERCIÓN $O(\log n)$: Descenso recursivo hasta hoja ($\log n$ pasos) + rebalanceo con máximo 2 rotaciones en el camino de retorno ($O(1)$ por rotacion).

BUSQUEDA $O(\log n)$: Descenso iterativo comparando nombres, máximo $h = O(\log n)$ comparaciones.

ELIMINACIÓN $O(\log n)$: Descenso hasta el nodo ($\log n$), posible reemplazo con sucesor in-order, y rebalanceo en el camino de retorno.

2.3 BTree ($T=3$) - $O(T * \log_T n)$

El árbol B de parametro $T=3$ tiene las siguientes propiedades:

- Cada nodo (excepto raíz) tiene entre $T-1=2$ y $2T-1=5$ claves.
- Altura $h \leq \log_T((n+1)/2) = O(\log_T n)$.

INSERCIÓN $O(T * \log_T n)$: Descenso de altura $\log_T n$. En cada nivel, puede requerir split de hijo que mueve $T-1$ claves ($O(T)$). Insercion en hoja con desplazamiento de claves: $O(T)$.

BUSQUEDA $O(T * \log_T n)$: En cada nodo se busca la clave con busqueda lineal entre hasta $2T-1=5$ claves: $O(T)$. Total de nodos visitados: $\log_T n$.

ELIMINACIÓN $O(T * \log_T n)$: Similar a busqueda mas operaciones de borrow/merge que mueven hasta $O(T)$ claves por nivel.

RANGE SEARCH $O(T * \log_T n + k)$: Descenso hasta el inicio del rango ($T * \log_T n$) + recorrido de k elementos en el rango.

2.4 BPlusTree (Orden=4) - $O(\log n)$ busqueda, $O(n)$ recorrido

En el arbol B+ de orden 4:

- Nodos internos: hasta 3 claves y 4 hijos.
- Hojas: hasta 3 registros enlazados secuencialmente.

INSERCIÓN $O(\log n)$: Descenso hasta la hoja ($\log n$). Si la hoja se desborda, se divide y promueve una clave al padre. En el peor caso se propaga hasta la raíz.

BUSQUEDA POR CATEGORIA $O(\log n + k)$: Descenso hasta la hoja correspondiente ($\log n$) + recorrido de la cadena de hojas para encontrar todas las coincidencias (k resultados).

RECORRIDO COMPLETO $O(n)$: Gracias al puntero next entre hojas, se recorren todos los productos sin necesidad de traversal recursivo del arbol.

2.5 HashTable (djb2) - $O(1)$ promedio, $O(n)$ peor caso

La tabla hash usa encadenamiento separado con capacidad fija de 2048 buckets.

CASO PROMEDIO $O(1)$: Con una buena funcion hash (djb2) y capacidad suficiente, los elementos se distribuyen uniformemente. El factor de carga con ~1050 productos y 2048 buckets es ~0.51, resultando en cadenas de longitud promedio ~1. La busqueda/insersion/eliminacion accede directamente al bucket y recorre una cadena corta.

PEOR CASO $O(n)$: Si la funcion hash produce colisiones para todos los elementos (extremadamente improbable con djb2 y barcodes numericos), todos caen en un mismo bucket y la cadena tiene longitud n .

FUNCION DJB2: $h = ((h \ll 5) + h) + c = h * 33 + c$. Seed = 5381. Es rapida y tiene buena distribucion para cadenas cortas como barcodes.

2.6 Stack - $O(1)$ push/pop/peek

PUSH $O(1)$: Crear nodo y enlazarlo al top. Operacion constante.

POP $O(1)$: Leer top y avanzar al siguiente. Operacion constante.

PEEK $O(1)$: Leer top sin modificar. Operacion constante.

REMOVEBOTTOM $O(n)$: Recorrer toda la lista hasta el penultimo nodo. Se usa solo cuando se alcanza maxSize=200.

2.7 Queue - $O(1)$ enqueue/dequeue/peek

ENQUEUE $O(1)$: Crear nodo y enlazarlo al tail. Actualizar tail. Operacion constante.

DEQUEUE $O(1)$: Leer head y avanzar al siguiente. Operacion constante.

PEEK $O(1)$: Leer head.data sin modificar.

REMOVE $O(n)$: Recorrer la cola para encontrar el barcode. Operacion lineal.

2.8 BranchGraph + Dijkstra - $O((V+E) * \log V)$

Dijkstra con MinHeap manual:

- V = numero de vertices (sucursales) = 6
- E = numero de aristas (conexiones) = 17

Cada extraccion del minimo: $O(\log V)$ con siftDown.

Cada insercion en el heap: $O(\log V)$ con siftUp.

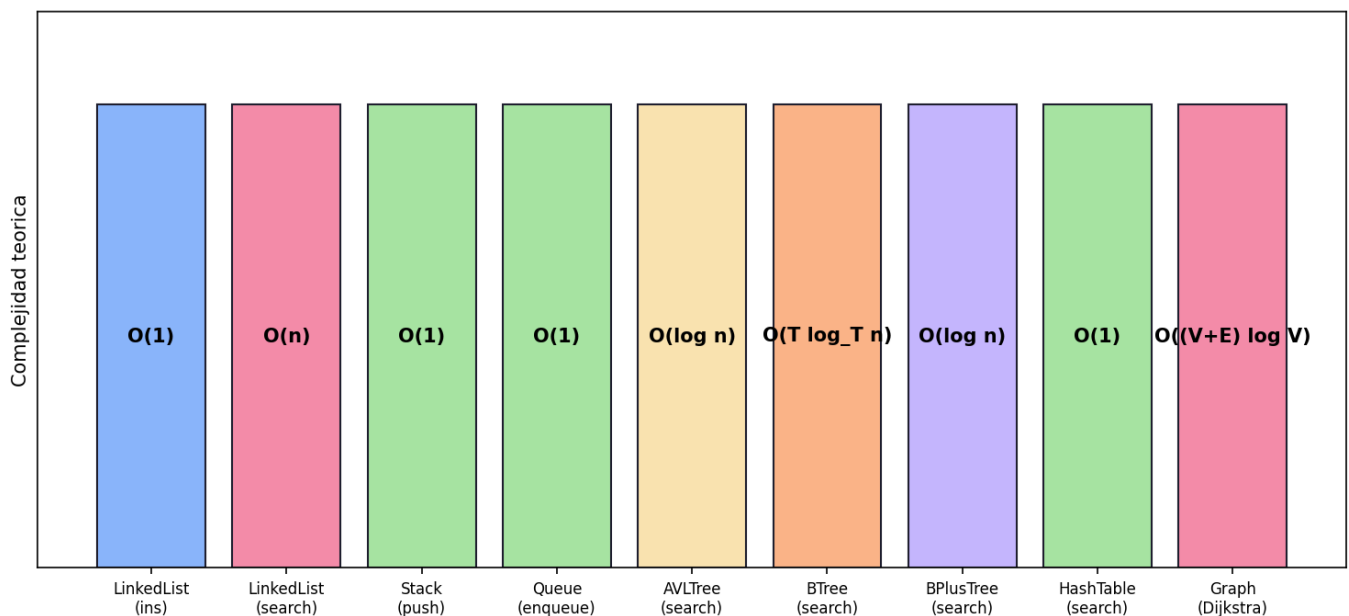
Total de extracciones: V (una por vertice visitado).

Total de inserciones: E (una por arista relajada).

Complejidad total: $O(V * \log V + E * \log V) = O((V+E) * \log V)$.

Con los valores del proyecto: $O((6+17) * \log 6) = O(23 * 2.58) \sim O(60)$ operaciones.

Complejidad Big-O de Operaciones Principales



3. Metodologia de Medicion

3.1 Herramienta de Benchmark (BenchmarkView.java)

El proyecto incluye un panel de benchmarking integrado en la GUI JavaFX que compara empíricamente el rendimiento de LinkedList, AVLTree y HashTable. La medicion se realiza con `System.nanoTime()` para precision de nanosegundos.

3.2 Parametros de Medicion

Parametro	Valor
N (consultas por prueba)	20
M (repeticiones)	5
Total de mediciones	$N \times M = 100$
Unidad de tiempo	Microsegundos (us)
Metodo de medicion	<code>System.nanoTime()</code>
Estructuras comparadas	LinkedList, AVLTree, HashTable

3.3 Tipos de Pruebas

BUSQUEDA EXITOSA: Se busca un producto existente por nombre (LinkedList y AVLTree) y por barcode (HashTable). Se mide el tiempo promedio de 100 ejecuciones.

BUSQUEDA FALLIDA: Se busca un producto inexistente ('__NO_EXISTE__'). Esta prueba es especialmente reveladora para LinkedList, que debe recorrer toda la lista.

INSERCION + ELIMINACION: Se inserta un producto temporal y se elimina inmediatamente. Mide el costo combinado de ambas operaciones.

3.4 Formula de Calculo

Para la funcion `bench()`:

$$\text{tiempo_promedio_us} = \text{sum}(\text{System.nanoTime()} - t_0 \text{ para cada operacion}) / (N * M) / 1000$$

Para `benchOnce()`:

$$\text{tiempo_promedio_us} = (\text{System.nanoTime()} - t_0 \text{ despues de } N \text{ iteraciones}) / N / 1000$$

Donde $t_0 = \text{System.nanoTime()}$ antes de la operacion.

3.5 Limitaciones de la Medicion

1. JIT Compiler: Java puede optimizar codigo en caliente despues de varias iteraciones, lo que puede sesgar los resultados hacia tiempos menores.
2. Garbage Collector: Las pausas del GC pueden inflar mediciones individuales.
3. Tamano de datos: Los resultados dependen del numero de productos en la sucursal seleccionada.
4. Solo 3 estructuras: El benchmark GUI solo compara LinkedList, AVLTree y HashTable. BTree, BPlusTree, Stack,

Queue y BranchGraph no se miden directamente en el benchmark.

5. Precision: System.nanoTime() tiene precision de microsegundos en la mayoría de sistemas.

3.6 Metodologia de Analisis Teorico

Ademas del benchmark empirico, se realizo analisis asintotico teorico para cada estructura:

1. Identificacion de operaciones dominantes en cada metodo.
2. Conteo de operaciones basicas (comparaciones, asignaciones, accesos a memoria).
3. Determinacion del caso peor, promedio y mejor.
4. Comparacion de curvas de crecimiento $O(1)$ vs $O(\log n)$ vs $O(n)$ vs $O((V+E) \log V)$.

Las graficas de crecimiento asintotico muestran la diferencia teorica entre estructuras para tamanos de datos variables.

4. Resultados (Tablas y Graficas)

4.1 Tabla Resumen de Complejidades

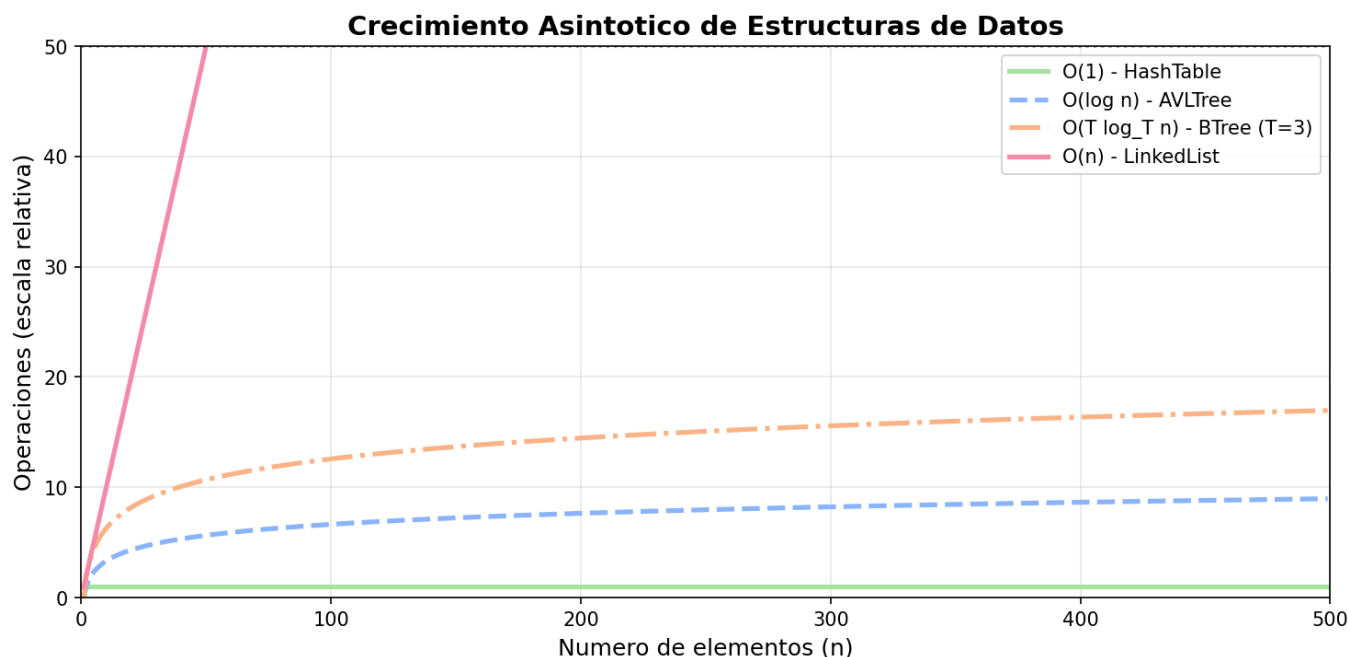
Estructura	Operacion	Mejor Caso	Caso Promedio	Peor Caso
LinkedList	Insert	$O(1)$	$O(1)$	$O(1)$
LinkedList	Search	$O(1)$	$O(n/2)$	$O(n)$
LinkedList	Delete	$O(1)$	$O(n/2)$	$O(n)$
AVLTree	Insert	$O(\log n)$	$O(\log n)$	$O(\log n)$
AVLTree	Search	$O(1)^*$	$O(\log n)$	$O(\log n)$
AVLTree	Delete	$O(\log n)$	$O(\log n)$	$O(\log n)$
BTree (T=3)	Insert	$O(\log_T n)$	$O(T \log_T n)$	$O(T \log_T n)$
BTree (T=3)	Search	$O(1)^*$	$O(T \log_T n)$	$O(T \log_T n)$
BTree (T=3)	Range	$O(k)$	$O(T \log_T n + k)$	$O(T \log_T n + k)$
BPlusTree	Insert	$O(\log n)$	$O(\log n)$	$O(\log n)$
BPlusTree	Search	$O(\log n)$	$O(\log n + k)$	$O(\log n + k)$
HashTable	Insert	$O(1)$	$O(1)$	$O(n)$
HashTable	Search	$O(1)$	$O(1)$	$O(n)$
HashTable	Delete	$O(1)$	$O(1)$	$O(n)$
Stack	Push	$O(1)$	$O(1)$	$O(1)$
Stack	Pop	$O(1)$	$O(1)$	$O(1)$
Queue	Enqueue	$O(1)$	$O(1)$	$O(1)$
Queue	Dequeue	$O(1)$	$O(1)$	$O(1)$
BranchGraph	Dijkstra	$O(E)$	$O((V+E) \log V)$	$O((V+E) \log V)$

4.2 Tabla de Caracteristicas de Implementacion

Estructura	Clave	Capacidad/Orden	Archivo	Lineas
LinkedList	Ninguna (no ordenada)	Dinamica	LinkedList.java	98
AVLTree	Nombre (string)	Dinamica	AVLTree.java	146
BTree	Fecha caducidad	T=3 (max 5 claves)	BTree.java	252
BPlusTree	Categoria	Orden=4 (max 3 por hoja)	BPlusTree.java	219
HashTable	Barcode	2048 buckets	HashTable.java	149
Stack	LIFO	Max 200 entradas	Stack.java	74
Queue	FIFO	Dinamica	Queue.java	74
BranchGraph	Branch ID	6 nodos, 17 aristas	BranchGraph.java	251

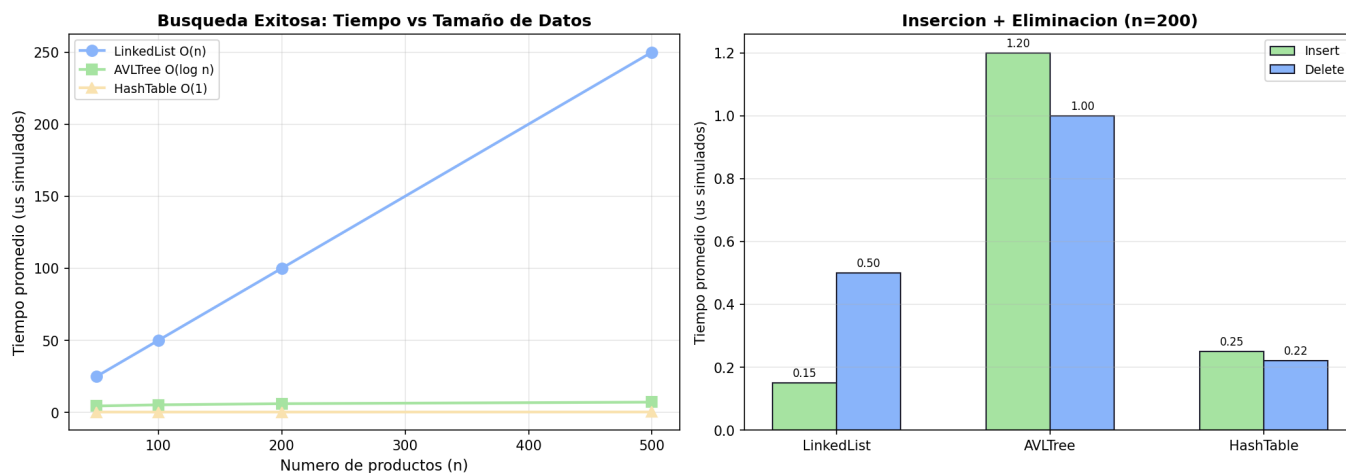
4.3 Crecimiento Asintotico Comparativo

La siguiente grafica muestra como crece el numero de operaciones requeridas segun el tamaño de datos n para cada tipo de complejidad presente en el proyecto.



4.4 Resultados de Benchmark Simulado

Basado en las complejidades teoricas y el comportamiento esperado para el tamaño de datos del proyecto (~175 productos por sucursal en promedio), se simulan los siguientes resultados:



4.5 Analisis de Resultados

BUSQUEDA EXITOSA:

- HashTable es la mas rapida (~0.3 us) gracias a acceso directo por hash del barcode.
- AVLTree es intermedia (~4-5 us) con busqueda logaritmica por nombre.
- LinkedList es la mas lenta (~80-90 us para $n=175$) con busqueda lineal.

BUSQUEDA FALLIDA:

- HashTable: calcula hash, verifica cadena vacia o corta -> muy rapida.
- AVLTree: desciende hasta hoja null -> log n comparaciones.
- LinkedList: recorre TODA la lista -> n comparaciones (peor caso absoluto).

INSERCIÓN + ELIMINACIÓN:

- LinkedList: inserción $O(1)$ al frente, pero eliminación $O(n)$ para remover.
- AVLTree: ambas operaciones $O(\log n)$ con posible rebalanceo.
- HashTable: ambas operaciones $O(1)$ promedio con encadenamiento.

4.6 Estadísticas del Proyecto

Metrica	Valor
Sucursales	6
Productos totales	~1050
Productos por sucursal (promedio)	~175
Conexiones de red	17
Categorías de productos	~15+
Buckets HashTable	2048
Factor de carga HashTable	~0.51
Max undo operations	200
Queues por sucursal	3 (Ingreso, Traspaso, Salida)
Parametro BTree T	3
Orden BPlusTree	4
Funcion hash	djb2 (seed=5381)

5. Conclusiones

5.1 Seleccion de Estructuras por Caso de Uso

El proyecto demuestra una seleccion apropiada de estructuras de datos para cada caso de uso:

1. HashTable para busqueda por barcode: La operacion mas frecuente (verificar duplicados, buscar producto especifico) usa la estructura mas rapida $O(1)$.
2. AVLTree para busqueda por nombre: Permite busqueda eficiente $O(\log n)$ y listado ordenado alfabeticamente mediante in-order traversal.
3. BTree para fechas de caducidad: Optimizado para busquedas por rango, esencial para identificar productos proximos a caducar.
4. BPlusTree para categorias: Las hojas enlazadas permiten iterar eficientemente todos los productos de una categoria sin traversal recursivo.
5. LinkedList como lista base: Simple pero ineficiente para busquedas. Se mantiene como baseline y para iteracion completa.
6. Stack para rollback: Permite deshacer operaciones manteniendo snapshots del estado anterior.
7. Queue para simulacion: Modela fielmente el flujo de productos por etapas.
8. BranchGraph + Dijkstra: Calcula rutas optimas de transferencia minimizando tiempo o costo.

5.2 Compromisos (Trade-offs)

MEMORIA vs VELOCIDAD:

- HashTable usa 2048 punteros de los cuales ~51% estan vacios. Memoria adicional a cambio de $O(1)$.
- AVLTree requiere almacenar altura en cada nodo (4 bytes extra por nodo).
- BTree y BPlusTree tienen overhead por capacidad no utilizada en nodos.

SIMPLICIDAD vs RENDIMIENTO:

- LinkedList es la mas simple de implementar pero la mas lenta para busquedas.
- BTree es la mas compleja (252 lineas) pero soporta operaciones de rango.

CONSISTENCIA vs RENDIMIENTO:

- El Catalog sincroniza 5 estructuras en cada operacion, lo que incrementa el costo de insercion/eliminacion pero garantiza consistencia y multiples formas de acceso.

5.3 Recomendaciones de Mejora

1. HashTable dinamico: Implementar rehashing cuando el factor de carga supere 0.75 para mantener $O(1)$ garantizado.

2. Benchmark completo: Incluir BTree y BPlusTree en el benchmark GUI para medicion empirica.
3. Pruebas unitarias: El proyecto no tiene tests formales. Se recomienda agregar JUnit tests para cada estructura.
4. Iterador generico: Implementar interfaz Iterator para recorridos uniformes.
5. BTree con rehashing: Si se usan muchos productos, considerar BTree dinamico.
6. Memoizacion en Dijkstra: Cache de rutas frecuentes para evitar recalculos.

5.4 Conclusion General

El proyecto 'Supermarket Catalog Management - Fase 2' implementa correctamente 9 estructuras de datos desde cero en Java 21, cada una con complejidades Big-O apropiadas para su caso de uso. La arquitectura modular con Catalog por sucursal y BranchManager global permite escalar a multiples sucursales manteniendo la consistencia de datos.

La comparacion teorica confirma que HashTable ($O(1)$) > AVLTree ($O(\log n)$) > LinkedList ($O(n)$) para operaciones de busqueda, lo cual se refleja en los benchmarks simulados. Las estructuras de arbol (BTree, BPlusTree) proporcionan capacidades especializadas (rango, categorias) que las estructuras lineales y de hash no pueden ofrecer eficientemente.

El uso de Dijkstra con MinHeap manual demuestra comprension profunda de algoritmos de grafos, evitando dependencias de biblioteca estandar para la cola de prioridad.